Sesión web scraping

Sergi Pons

Objetivo Principal

Proporcionar las herramientas esenciales para realizar web scraping de forma autónoma y superar las barreras iniciales de la programación.

Objetivos Específicos

1. **Familiarización con el Entorno de Desarrollo**
    - Configuración del entorno de trabajo
    - Introducción a Jupyter Notebook
    - Gestión de paquetes y dependencias
2. **Dominio de Herramientas Básicas**
    - Jupyter Notebook como entorno interactivo
    - Pandas para manipulación de datos
    - BeautifulSoup para procesamiento HTML
3. **Habilidades Prácticas**
    - Extracción de datos estructurados
    - Automatización de procesos de recolección de datos
    - Análisis básico de datos web

Estructura de la sesión

- Explicar web scraping y sus implicaciones
- Qué es Python y por que su uso extendido
- Principales librerías de Python para Scraping
- Consideraciones antes de hacer scraping
- Proceso general del web scraping
- Entornos de desarrollo
- Comandos principales y shortcuts
- Recursos y materiales (fuentes públicas)
- Git y creación de entorno
- Primeros pasos de programación con python

Web Scraping

Definición

- Técnica de extracción automatizada de datos de sitios web
- Proceso de recolección de información estructurada de páginas web
- Transformación de datos no estructurados en datos estructurados

Usos Comunes

- Investigación de mercado
- Agregación de noticias
- Análisis de sentimientos
- Machine Learning y análisis de datos

Técnicas Principales

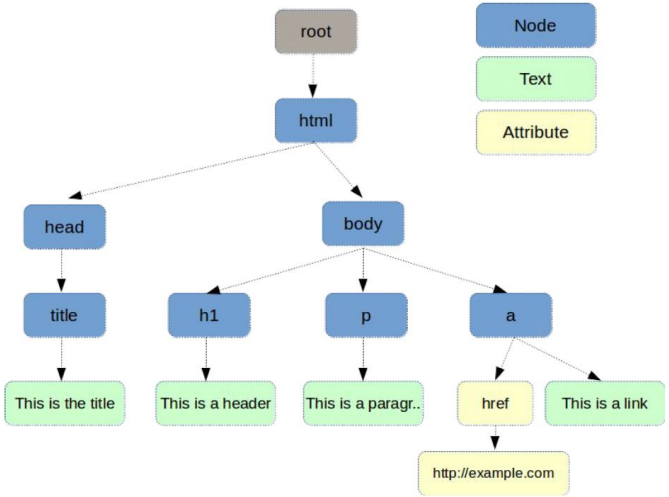
Parsing HTML

- Análisis del código fuente HTML
- Uso de selectores CSS y XPath
- Extracción de datos mediante DOM (Document Object Modelling)

APIs

- Uso de APIs públicas (INE)
- REST APIs (Google maps, spotify…)
- API Wrappers (biblioteca que simplifica tareas en la interacción con la API, como Tweepy)

```html
<!DOCTYPE html>
<html>
    <head>
        <title>This is the title</title>
    </head>
    <body>
        <h1>This is a header</h1>
        <p>This is a paragraph</p>
        <a href="http://example.com">This is a link</a>
    </body>
</html>
```



Python

Python es un lenguaje de programación interpretado que fue creado por Guido van Rossum en 1991 y que persigue una sintaxis que favorezca un código legible.

Características principales

- Fácil de aprender
- Con una gran cantidad de librerías disponibles
- Interactivo
- Multiplataforma
- De código libre
- Interpretado
- Orientado a Objetos

fuente: Sesión web scraping Carles Illa (2018)

Herramientas Principales

Librerías Python

- **Requests**
  - Peticiones HTTP
  - Manejo de sesiones
  - Headers personalizados
- **Beautiful Soup 4**
  - Parser HTML/XML
  - Fácil de usar
  - Ideal para páginas estáticas
- **Scrapy**
  - Framework open source
  - Escalable
  - Automatiza la navegación entre páginas
- **Selenium**
  - Puede usarse en páginas que usan JavaScript
  - Interacción como clics o rellenados como si fuera usuario real

4. Consideraciones Importantes

4. Aspectos Legales

- Términos de servicio
- Robots.txt
- Derechos de autor
- Políticas de privacidad

```
User-agent: *
Disallow: /bc/

User-agent: Twitterbot
Allow: /bc/*

User-agent: facebookexternalhit
Allow: /bc/*

User-agent: GPTBot
Disallow: /

User-agent: ClaudeBot
Disallow: /

User-agent: anthropic-ai
Disallow: /

User-agent: Claude-Web
Disallow: /
```

1. <u>Para todos los bots (*):</u>

   ❌ No pueden acceder a la sección `/bc/` (ejemplo: contenido exclusivo o privado).

2. <u>Bots de redes sociales (Twitterbot, Facebook):</u>

   ✅ Sí pueden acceder a `/bc/*` (probablemente para compartir enlaces en sus plataformas).

3. <u>Bots de inteligencia artificial (GPTBot, ClaudeBot...):</u>

   ❌ Bloqueados totalmente (no pueden rastrear ninguna parte del sitio).

Librerías y frameworks populares

- **Requests**, HTTP for Humans
- **Scrapy**, a Fast and Powerful Scraping and Web Crawling Framework
- NumPy, the fundamental package for scientific computing with Python
- SciPy, a library of algorithms and mathematical tools for python
- **Pandas**, high-performance, easy-to-use data structures and data analysis tools for Python
- Matplotlib, python data visualization
- SQLAlchemy, the Database Toolkit for Python
- **Jupyter Notebook**, an interactive command-line terminal for Python
- Django, a high-level Python Web framework
- NLTK, a leading platform for building Python programs to work with human language data

fuente: Sesión web scraping Carles Illa (2018)

Proceso general de Web Scraping

1. Obtención del contenido HTML de la página web
2. Extracción de la información (parsing)
3. Transformación
4. Almacenamiento
5. Ir a otra página web y repetir el proceso (crawling)

fuente: Sesión web scraping Carles Illa (2018)

Almacenamiento y conexión de datos

1. Formatos de almacenamiento
   - CSV (Comma Separated Values) para datos de texto.
   - JSON (JavaScript Object Notation) para estructuras complejas.

2. Creación de una API: Facilitar el acceso a los datos.
3. Repositorios de datos: Kaggle, UCI Machine Learning Repository, Github, Data World

Google Colab

Exchange rates API GETting a JSON response.ipynb

Archivo Editar Ver Insertar Entorno de ejecución Herramientas Ayuda

Compartir

+ Código + Texto

RAM Disco

Gemini

https://manage.exchangeratesapi.io/login

[1]
```
# loading the packages
# requests provides us with the capabilities of sending an HTTP request to a server
import requests
```

Extracting data on currency exchange rates

[2]
```
# We will use an API containing currency exchange rates as published by the European Central Bank
# Documentation at https://exchangeratesapi.io
```

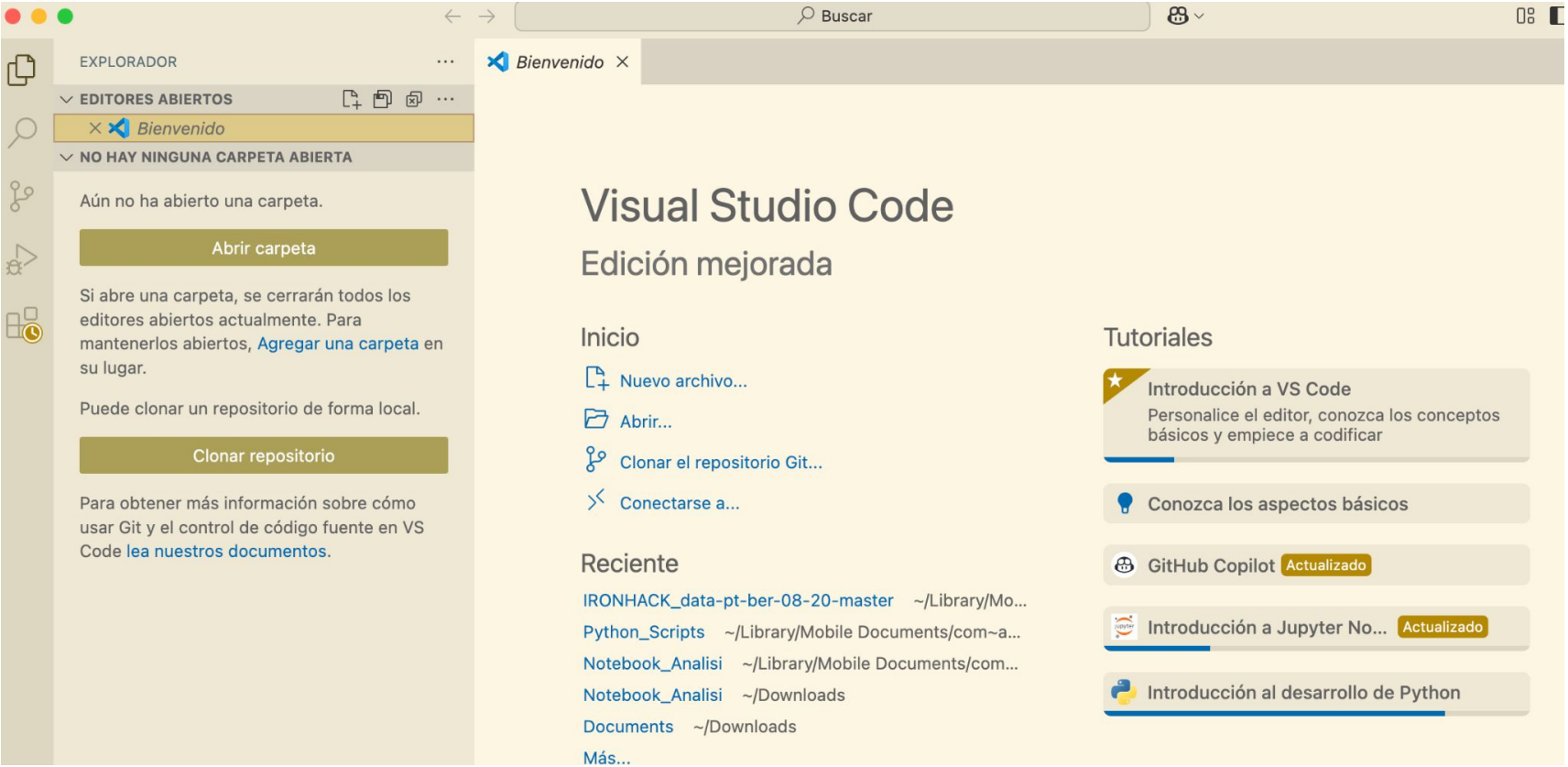
Sending a GET request

[3]
```
# Define the base URL
# Base URL: the part of the URL common to all requests, not containing the parameters
base_url = "https://api.exchangeratesapi.io/latest"
```

[10]
```
# Tu clave de API
api_key = '2a5431570440f3e9f39cea4b249264ac'

# URL base de la API
base_url = 'https://api.exchangeratesapi.io/v1/latest'
```

Visual Studio Code



Jupyter Notebook

jupyter SesiónWebScraping Last Checkpoint: 13 days ago

File Edit View Run Kernel Settings Help Trusted

Markdown JupyterLab Python 3 (ipykernel)

o categorías.

- Diccionarios: Los diccionarios permiten almacenar datos en pares clave-valor, lo cual es ideal para manejar información estructurada, como datos de un producto.
- Strings: cadenas de texto
- Integer y floats: números

Listas

[2]:
```
# Una lista con nombres de productos
productos = ["Cámara", "Teléfono", "Laptop", "Audífonos"]

# Acceder al primer elemento
print("Primer producto:", productos[0])

# Recorrer todos los elementos
for producto in productos:
    print("Producto:", producto)
```

```
Primer producto: Cámara
Producto: Cámara
Producto: Teléfono
Producto: Laptop
Producto: Audífonos
```

tuplas

Comandos terminal:

`python --version` → Muestra la versión de Python instalada.

`conda list` → Muestra los paquetes instalados en el entorno activo.

`pip list` → Muestra una lista de paquetes instalados con `pip`.

`pip install nombre_paquete` → Instala un paquete con `pip`.

`pip install --upgrade pip` → Actualiza un paquete con `pip`.

`pip freeze > requirements.txt` → Guarda la lista de paquetes instalados en un archivo `requirements.txt`.

`pip install -r requirements.txt` → Instala paquetes desde un archivo `requirements.txt`.

`which python` o **`where python`** (en Windows) → Muestra la ruta del intérprete de Python activo.

`which jupyter` → Muestra la ruta donde está instalado Jupyter.

`conda clean --all`

Trucos/shortcuts:

`Ctrl + Enter` → Ejecuta la celda sin moverse a la siguiente.

`Alt + Enter` → Ejecuta la celda y crea una nueva debajo.

`A` → Inserta una celda arriba.

`B` → Inserta una celda abajo.

`M` → Convierte la celda en Markdown.

`Y` → Convierte la celda en código.

`D D` → Elimina la celda seleccionada.

`Z` → Deshace la eliminación de una celda.

`Shift + M` → Fusiona la celda actual con la siguiente.

`Ctrl + S` → Guarda el notebook.

Tipos de estructuras de datos para manejar la información

- Listas: son estructuras de datos en Python que permiten almacenar múltiples elementos en un orden específico. Son útiles para manejar datos como listas de productos, títulos de artículos, o enlaces.
- Tuplas: Las tuplas son como listas, pero no se pueden modificar después de su creación. Son útiles para representar datos que no deben cambiar, como coordenadas o fechas.
- Sets (conjuntos de datos): Los conjuntos son útiles cuando necesitas asegurarte de que no haya duplicados en tu colección de datos, como en una lista de etiquetas o categorías.
- Diccionarios: Los diccionarios permiten almacenar datos en pares clave-valor, lo cual es ideal para manejar información estructurada, como datos de un producto.
- Strings: cadenas de texto
- Integer y floats: números

Recursos

Kaggle: https://www.kaggle.com/datasets

Repositorios github:

https://github.com/alabarga/pandas-workshop

https://github.com/alabarga/DataScienceWorkshop-BS/tree/master

https://github.com/Pierian-Data/Complete-Python-3-Bootcamp

https://www.w3schools.com/python/default.asp

Configuración Inicial del proyecto en escritorio

1. Preparación del Directorio

Creación del Directorio del Proyecto

```
# Navegar al escritorio
cd ~/Desktop

# Crear carpeta del proyecto
mkdir proyectoWS

# Entrar en la carpeta
cd proyectoWS
```

Configuración Inicial del proyecto en escritorio

2. Configuración del Entorno Virtual

Creación y Activación

```
# Crear entorno virtual
python3 -m venv env

# Activar el entorno virtual
source env/bin/activate
```

Notas Importantes

- El prompt mostrará `(env)` cuando el entorno esté activo
- Para desactivar: `deactivate`
- El entorno virtual ayuda a mantener las dependencias aisladas

Configuración Inicial del proyecto en escritorio

3. Inicialización del Control de Versiones

Configuración de Git

```
# Inicializar repositorio
git init

# Crear README inicial
echo "# 1r proyectoP" >> README.md

# Primer commit
git add README.md
git commit -m "Primer commit: archivo inicial"
```

4. Buenas Prácticas

- Siempre activar el entorno virtual antes de trabajar
- Mantener actualizado el README.md
- Hacer commits frecuentes y con mensajes descriptivos
- No incluir el directorio `env/` en el control de versiones

5. Próximos Pasos

1. Crear `.gitignore` para excluir archivos innecesarios
2. Configurar dependencias del proyecto
3. Establecer estructura de carpetas del proyecto
4. Documentar instalación y configuración en README.md